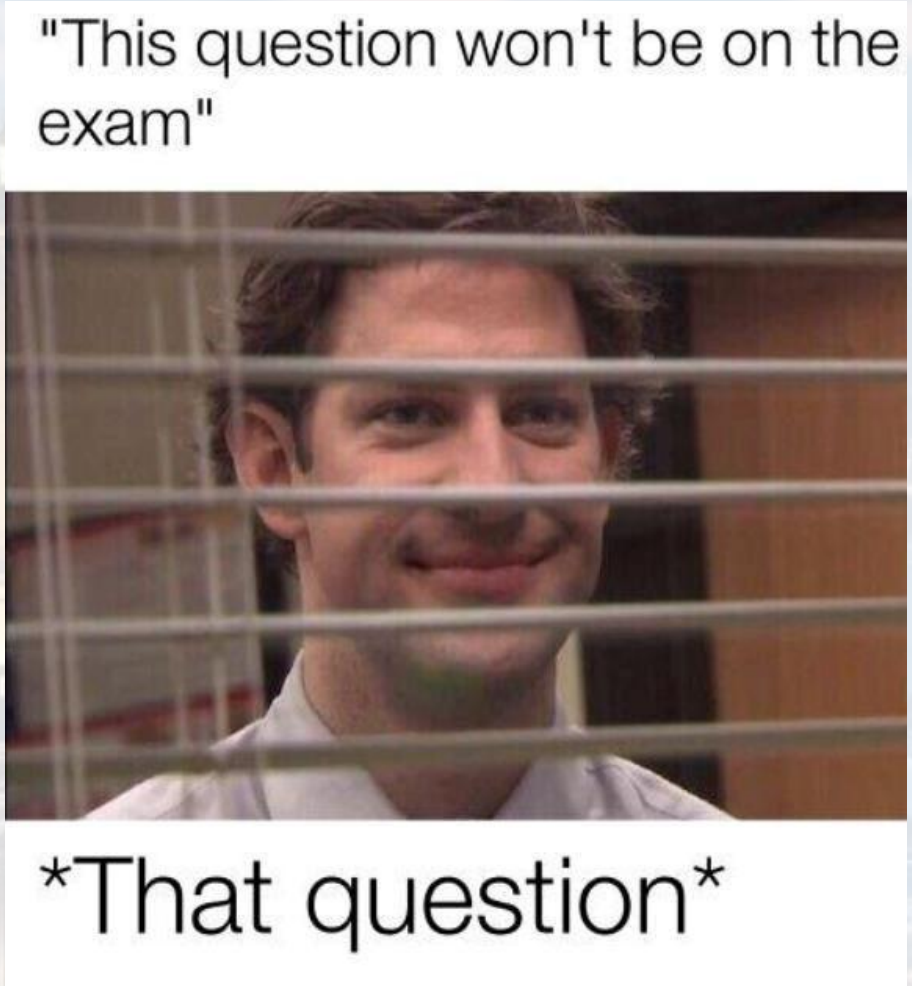
Chem 277A: Project II



Project II

Instructions: - today

Due: - **May 9th**

Structure: - **Collaborative** Final Project, in lieu of a final exam
→ form teams of **2-4** by **Apr 13th** here

A	B	C	D	E	F	G	H	I	J	K	L	M
deadline: Apr 13th												
	Team 1	Team 2	Team 3	Team 4	Team 5	Team 6	Team 7	Team 8	Team 9	Team 10	Team 11	Team 12

Project II

Deliverables:

- pick a dataset of your choice
- data set has to be one, which we **haven't used in the course**
- data set has to be a **real data set**
- feature encoding
- feature selection (eg. PCA)
- run **at least three** methods (see also next modules), regression or classification and compare performance
- collaboration via version control Git/GitHub recommended, but not needed
- **each team** submits a **Jupyter Notebook** by **May 9th**, that clearly documents each step of the analysis

Project II

title and group members

describe the problem

libraries

EDA, feature encoding, selection etc

Walk Through L1 and L2

1) Objective

We learned in the lecture that two common regularization methods are L1 (aka "LASSO") and L2 (aka "RIDGE"), where L1 encourages sparcity and reduces overfitting and L2 penalizes large magnitudes of the coefficients β.
In this example, we want to fit a logistic model to the diabetes data set we know from the Lab Course Module 6 and compare the result to the best result we obtain by implementing regularisation.

2) Calling Libaries

For visualizations, for scanning the model for the best L1 and L2 parameters (Lagrangian Multiplier) and for bootstrapping (see later) we will use pre-defined custom functions which are stored in a file called `Auxiliary.py`. This file also calls all the standard libraries we are going to need. Therefore, we load `Auxiliary.py` and check its methods:

```
[ ]: import sys
     sys.path.insert(1, '../10 Codes')

     from Auxiliary import *
```

```
[ ]: help('Auxiliary')
```

3) Loading, Splitting and Scaling the Data

Next, we load the data set, separate `X` and `Y`, scale the data accordingly and split the set into training and test set.

```
[ ]: location = FindMyFile('Diabetes2.csv')
```

```
[ ]: Data = pd.read_csv(location, sep = ';')
     Data.head()
```

```
[ ]: X    = Data.drop(['label'], axis = 1)
     Y    = Data['label']
```

Since we want to compare the test accuarcy from the standard model to the test accuracy we obtain from the best regularization parameters, we are aiming on a larger test set so that the results are more reliable. Alternatively, we could also run a k-fold cross-validation test.
There are eight fit parameters, i. e. nine coefficients in total if we include the intercept. The data set contains almost 800 data points, i.e. even if we split training and test data in, say 60/40, the number of data points is more than an order of magnitude larger than the number of coefficients.

```
[ ]: X_Train, X_Test, Y_Train, Y_Test = train_test_split(X, Y, test_size = 0.40, random_state = 1)
```

Scaling the data:

```
[ ]: scaler       = StandardScaler()

     XS_Train     = scaler.fit_transform(X_Train)
     XS_Test      = scaler.transform(X_Test)
```

hint: use “`WalkThrough.ipynb`” as template

Project II

run at least three different methods

4) Standard Logistic Model

We first run the standard logistic model. The cross entropy plots, the accuracy and the confusion matrix will be the benchmark in order to compare it to the models with regularization.

```
[ ]: #adding the intercept to X and creating dummy variables for Y:
XS_Train_add = sm.add_constant(pd.DataFrame(XS_Train, columns = X.columns))
Ydumm        = pd.get_dummies(Y_Train).reset_index(drop=True)
```

```
[ ]: #the fit:
my_model = sm.GLM(Ydumm, XS_Train_add, family = sm.families.Binomial())
result   = my_model.fit()
result.summary()
```

We see that some of the coefficients (like e. g. for *SkinThickness*) have a large p-value and that their values are consitent with zero. Nevertheless, we evaluate the fit quality by calculating the **accuracy**:

```
[ ]: Probs  = result.predict(sm.add_constant(XS_Test))
ProbsR = np.round(Probs)
Ypred  = ['diabetes' if p== 1 else 'healthy'for p in ProbsR]
```

```
[ ]: acc = 100*sum(Ypred == Y_Test)/len(Y_Test)
print(f"Accuracy is {acc: .2f}%")
```

Next, we plot the **confusion matrix**:

```
[ ]: plot_confusion(Y_Test, Ypred,['diabetes', 'healthy'])
```

and create the **cross-entropy** plot

```
[ ]: Pmatrix = np.vstack((Probs, 1 - Probs)).transpose()
plot_entropy(Pmatrix, [1 if y== 'diabetes' else 0 for y in  Y_Test], ['diabetes', 'healthy'], [1, 0])
```

As we can see, the model is very confident in voting for *"healthy"*, but essentialy just guesses when voting for *"diabetes"*. Note also the inbalance of the different classes.

apply improvements, discuss performance

5) Regularization

In order to run a fit with regularization we need to run `my_model.fit_regularized` once we initialized `my_model`. The standard method is `'elastic_net'` which provides a convenient way to balance both regularization terms. Let α_1 and α_2 be the regularization coefficients, then the Lagrangian $\mathcal{L}$ reads

$$\mathcal{L} = \tilde{\mathcal{L}} + \alpha_1 \left(\frac{1-\alpha_2}{2} \|\beta\|_2^2 + \alpha_2 \|\beta\|_1^1 \right)$$

where $\tilde{\mathcal{L}}$ is the Lagrangian/Loss without regularization, i. e. $\tilde{\mathcal{L}} = \|Y - X\beta\|_2^2$.
As defined by the elastic net equation, setting $\alpha_2 = 1$ equals L1 regularization and $\alpha_2 = 0$ equals L2 regularization. Therefore, the actual Lagrangian multiplier are $\lambda_1 = \alpha_1\alpha_2$ and $\lambda_2 = \alpha_1 \frac{1-\alpha_2}{2}$.

5.1) Test Fit

Next, we pick some values for α_1 and α_2 and see, if the result improves.

```
[ ]: alpha1 = 0.1
alpha2 = 0.1

result_reg = my_model.fit_regularized(method = 'elastic_net', alpha = alpha1, L1_wt = alpha2)

Probs  = result_reg.predict(sm.add_constant(XS_Test))
ProbsR = np.round(Probs)
```

hint: use "`WalkThrough.ipynb`" as template

apply improvements, discuss performance

Project II

no regularization

"sparse" model

	coef	std err	z	P>|z|	[0.025	0.975]
const	-0.8517	0.122	-6.966	0.000	-1.091	-0.612
Glucose	1.1150	0.133	8.398	0.000	0.855	1.375
BMI	0.7348	0.128	5.719	0.000	0.483	0.987

Accuracy is 75.97%

only those with small p-values "survived"

complete model

	coef	std err	z	P>|z|	[0.025	0.975]
const	-0.9067	0.128	-7.087	0.000	-1.157	-0.656
Pregnancies	0.2391	0.134	1.786	0.074	-0.023	0.501
Glucose	1.0384	0.152	6.834	0.000	0.741	1.336
BloodPressure	-0.2400	0.139	-1.728	0.084	-0.512	0.032
SkinThickness	-0.0690	0.151	-0.457	0.648	-0.365	0.227
Insulin	0.0839	0.145	0.581	0.562	-0.199	0.367
BMI	0.8625	0.166	5.210	0.000	0.538	1.187
DiabetesPedigreeFunction	0.1161	0.126	0.923	0.356	-0.130	0.363
Age	0.3368	0.145	2.330	0.020	0.053	0.620

Accuracy is 75.65%

L1 and L2 regularization

	conf 0.025	best	conf 0.975
const	-0.497288	-0.339920	-0.175653
Glucose	0.435954	0.598282	0.766013
BMI	0.117937	0.282437	0.431760

Accuracy is 77.92%

only those not consistent with zero "survived"

	conf 0.025	best	conf 0.975
const	-0.644043	-0.339928	-0.378195
Pregnancies	0.000000	0.000000	0.276625
Glucose	0.468804	0.598115	0.723600
BloodPressure	-0.132069	0.000000	0.074831
SkinThickness	0.000000	0.000000	0.207292
Insulin	0.000000	0.000000	0.356646
BMI	0.287730	0.282465	0.525576
DiabetesPedigreeFunction	-0.034453	0.000000	0.235977
Age	0.031497	0.000703	0.296114

Accuracy is 77.92%

Project II

goal:

typical workflow:

start:

```
>GBDP50119-19|Aedes africanus|COI-5P|MF183656
TTAAATTTCGATCTGTTAATAATATAGTAATAGCTCCTGCTAAAAC
ATTGCTAAAT
GAATCCTCCA
```

```
>GBMIN56476-17|Aedes albopictus|COI-5P|KY378921
GTTTTAATTCGTATTGAACTTAGACATCCTGGTATATTTATTGGAAATGA
TACTAGGAGCCCCTGATATAGCTTTTCCTCGAATAAATAATATAAGTTTT
TCATGCTGGGGCTTCAGTTGATTTAGCAATTTTTTCTTTACATTTAGCGG
GTAATTACAGCTATTTTATTACTTCTTTCTCTACCCGTATTAGCCGGAGC
```

raw data

data exploration (EDA)
- How many files?
- How big are they?
- How many sets?
- consistency
- homogeneity
- gaps? Missing data?

data extraction
- sampling
- filtering

feature selection I
- Which information is relevant?
- Which information is redundant?
- Is there missing information/ biases?

feature engineering
- normalization
- encoding
- interpolation/extrapolation

feature selection II
- correlation analysis

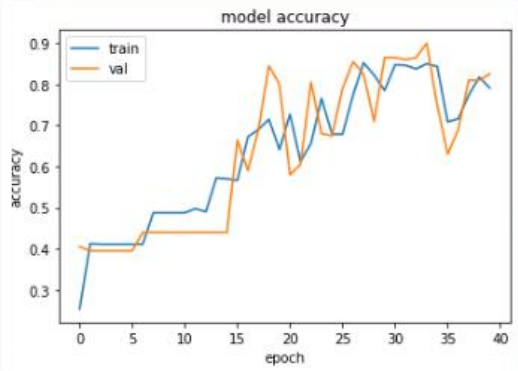
model selection

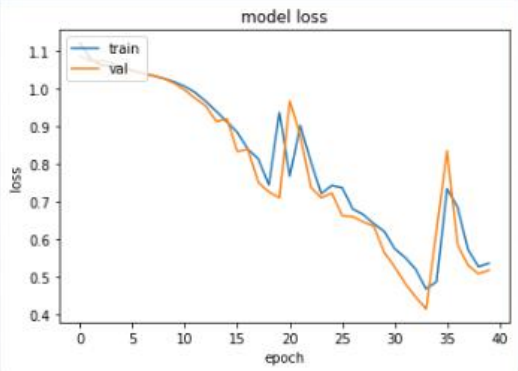
goal: classification/ regression/ prediction/ generation

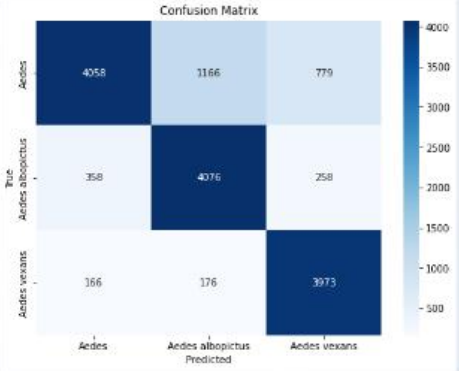
structure: timeseries/images/ sequences/perm invariance

feature selection III
- significance
- "leave one out"

goal: a **model** that:
- well **describes** the data
- covers **relevant information** of the data set
- can be used for **predictions**
- ...







Project II

Jupyter Notebook:

- Introduction: What is the problem? Why is it important? Brief historical view.
  1 - 2 paragraphs (not counting figures)
- Methods: at least three algorithms/ models
  0.5 – 1 pages
- Implementation: How did you do it exactly.
  2 – 5 pages (not counting figures)
- Evaluation: Present results in a clear and coherent way, compare methods.
  2 – 5 pages (not counting figures)
- Discussion: Interpret results, discuss limits of your model and improvements
  1 page (not counting figures)
- Contribution: each group member lists what they contributed to project II
- References

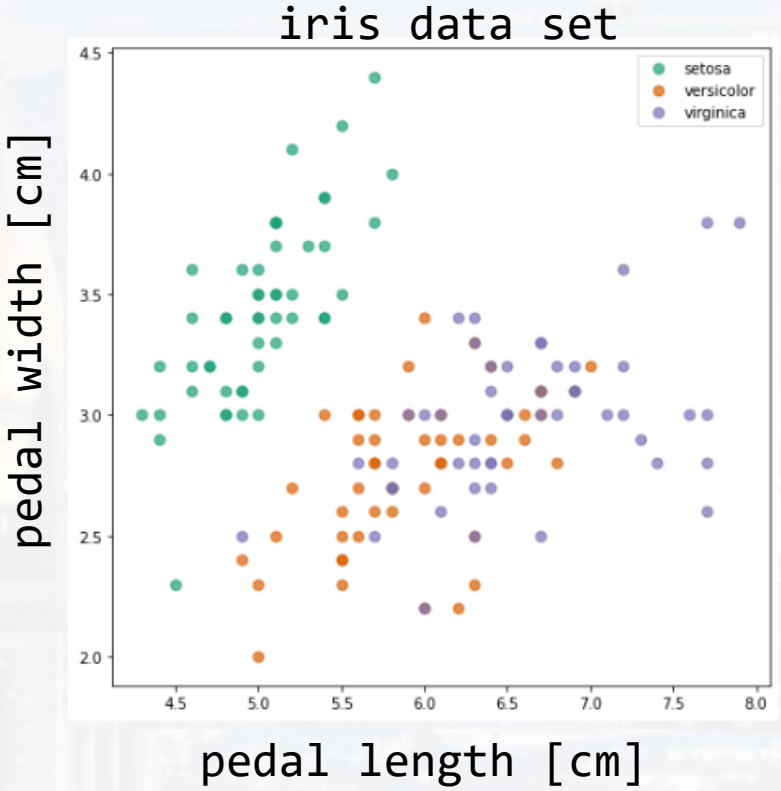
raw files (attachment):

- such that we can run the codes and reproduce your results

Project II

important: back-up **each step** with publication-ready visualizations!

- **t**itle
- **u**nits
- **l**abels
- **l**egends



Project II

Code of Honor:

- Don’t copy/paste someone else's **(including AI)** code without citing it!
- It should be **your(!)** work
- Cite, if you use **any** external source
- Each team member **contributes equally** to the project!
- Not following the code of honor is considered **cheating and will be graded as fail**

Example Data Sets:

- Gene Expression ALL/AML
- Toxicology
- RNA

kaggle PubMed

Important: Good datasets are hard to find!
Put effort and time in finding a suitable dataset – it will save time in the long run!

Project II

Good formatting!

Discovering Phenotype-Modules and Protein Binding Site in Alzheimer's using Graph-Transformer

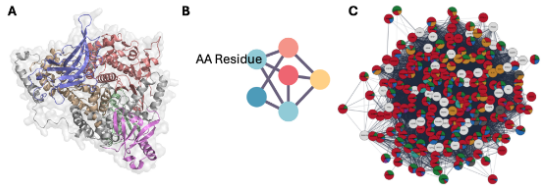
Group Members: Jingran Wang, Yufei Cai, Vincent Nguyen, Zi Fang

Skillsets we've learned this far:
Supervised learning: linear regression, OLS, Logistic regression
Unsupervised learning: k-mean, GMM … gradient descent

1. Project description:

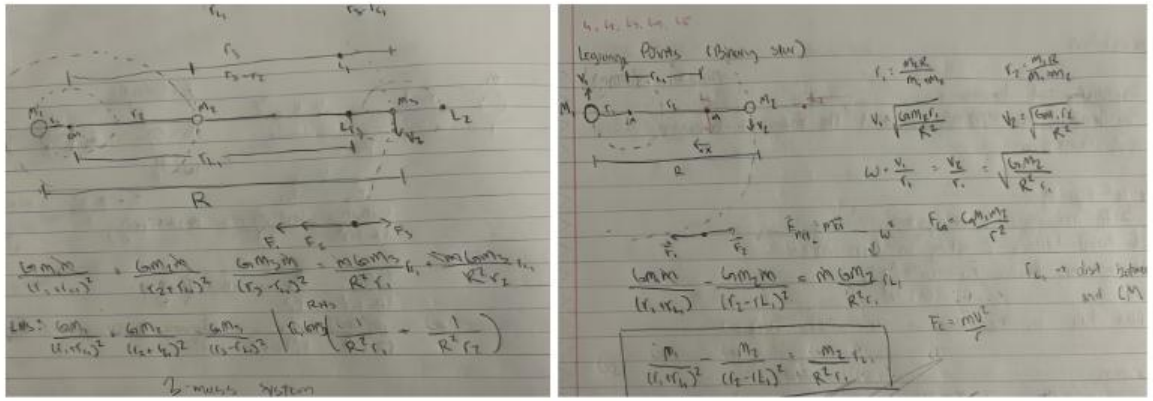
1.1 Introduction

Proteins are the carriers of life activities. Within a protein, an intramolecular network can be formed by representing amino acids as vertices and covalent/non-covalent bonds as edges. We can further extend these interaction networks to also simulate intermolecular networks between different proteins. The diagram below shows the Intermolecular and Protein-Protein Interactions Network:



Bad formatting!
→ will be graded poorly
→ or even not accepted

are located when the masses and points form an equilateral triangle, i.e., when the angle between each point is 60 degrees. Our derivations of these points are located below.



Project II

Need feedback for your project?

→ meet us during **office hours**

→ during next **lab courses**: teams work in breakout rooms
Lizzie and Markus are there to answer questions and help if you got stuck!

Questions?